SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	FURTHOSE SAMREEN B	**Reg. No.:**	192511164
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with			Poor presentation and difficult to

		logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: ___	Signature: ___	Signature: ___
Date: _	Date: _	Date: _

Q10. Huffman Coding vs Fixed-Length Coding

Given symbol frequencies {A:5, B:9, C:12, D:13, E:16, F:45}, construct Huffman codes and compare with fixed-

length encoding. Analyze compression efficiency and code length. Critically evaluate advantages and

limitations. Generate insights on real-world data compression.

Instructions: Solve the assigned problem, show the algorithm/steps clearly, analyze correctness, time and space complexity, and provide the requested comparison or insights.

Huffman Coding vs Fixed-Length Coding

1. Given Data

Symbol	Frequency
A	5
B	9
C	12
D	13
E	16
F	45
Total	**100**

---

2. Huffman Coding Algorithm

Huffman coding is a **greedy algorithm** that assigns shorter binary codes to more frequent symbols and longer codes to less frequent symbols.

Step 1: Arrange frequencies in ascending order

A(5), B(9), C(12), D(13), E(16), F(45)

Step 2: Combine the two smallest frequencies

- A(5) + B(9) = **14**

Now:

C(12), D(13), 14, E(16), F(45)

Step 3

- C(12) + D(13) = **25**

Now:

14, E(16), 25, F(45)

Step 4

- 14 + E(16) = **30**

Now:

25, 30, F(45)

Step 5

- 25 + 30 = **55**

Now:

F(45), 55

Step 6

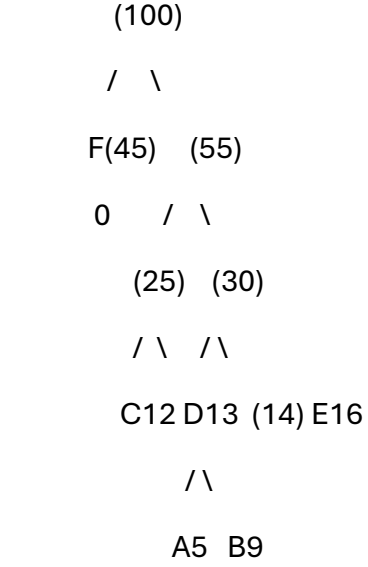
- 45 + 55 = **100**

The Huffman tree is complete.

---

3. Huffman Tree

One valid assignment of 0 and 1 gives:



Therefore, the Huffman codes are:

Symbol	Frequency	Huffman Code	Length
A	5	1100	4

Symbol	Frequency	Huffman Code	Length
B	9	1101	4
C	12	100	3
D	13	101	3
E	16	111	3
F	45	0	1

The exact 0/1 values can differ depending on how left/right branches are chosen, but the **code lengths and total cost remain the same**.

---

4. Calculate Huffman Encoding Size

The total number of bits is:

$$
(5\times4)+(9\times4)+(12\times3)+(13\times3)+(16\times3)+(45\times1)
$$

$$
=20+36+36+39+48+45
$$

$$
\boxed{224\ bits}
$$

Average number of bits per symbol:

$$
\frac{224}{100}=\boxed{2.24\ bits/symbol}
$$

---

5. Fixed-Length Coding

There are **6 symbols**.

For fixed-length coding:

$$
\lceil \log_2 6 \rceil = 3
$$

So, each symbol requires **3 bits**.

For example:

Symbol	Fixed-Length Code
A	000
B	001
C	010
D	011
E	100
F	101

For 100 symbols:

$$
100 \times 3 = \boxed{300\ \text{bits}}
$$

Average:

$$
\boxed{3\ \text{bits/symbol}}
$$

---

6. Comparison

Feature	Huffman Coding	Fixed-Length Coding
Number of symbols	6	6
Average bits/symbol	**2.24**	**3**
Total bits	**224**	**300**

Feature	Huffman Coding	Fixed-Length Coding
Bits saved	**76**	—
Compression	Better	Lower
Variable length	Yes	No
Uses frequency	Yes	No

Compression saving

$$
300-224=76\ \text{bits}
$$

Percentage saving:

$$
\frac{300-224}{300}\times100
$$

$$
=\boxed{25.33\%}
$$

Thus, Huffman coding reduces the storage requirement by approximately **25.33%** for this dataset.

---

7. Correctness Analysis

Huffman coding is correct because:

1. It repeatedly combines the two least frequent symbols.
2. Less frequent symbols are placed deeper in the tree.
3. More frequent symbols receive shorter codes.
4. The resulting codes are **prefix-free**, meaning no code is the prefix of another code.
5. Therefore, the encoded data can be decoded uniquely.

For example:

- F = 0
- C = 100
- D = 101

Since 0 is not the prefix of any other complete code, decoding is unambiguous.

---

8. Time Complexity

If a **min-priority queue (min-heap)** is used:

- Building the Huffman tree: **O(n log n)**
- Extracting and combining nodes: **O(n log n)**

Therefore:

[
\boxed{\text{Time Complexity = }O(n\log n)}
]

where n is the number of symbols.

Space Complexity

The Huffman tree requires:

[
\boxed{O(n)}
]

space.

---

9. Advantages of Huffman Coding

- Provides better compression than fixed-length coding when frequencies are unequal.
- Frequently occurring symbols receive shorter codes.
- It is **lossless**, so no information is lost.
- Codes are prefix-free and easy to decode.

- Useful for text and other data with non-uniform symbol frequencies.
- Forms an important part of compression techniques such as **DEFLATE**.

---

10. Limitations

- A frequency table or Huffman tree must also be stored/transmitted.
- For small datasets, this extra overhead can reduce the actual compression benefit.
- Encoding and decoding are more complex than fixed-length coding.
- If all symbols occur with almost equal frequency, Huffman coding provides little or no benefit.
- The code is dependent on the frequency distribution.

---

11. Real-World Insights

In real-world data, some symbols occur much more frequently than others. Huffman coding takes advantage of this uneven distribution.

For example, in this problem:

- F occurs **45 times** and receives only **1 bit**.
- A occurs only **5 times** and receives **4 bits**.

This demonstrates the main idea of Huffman coding:

Frequent symbols → shorter codes
Rare symbols → longer codes

Therefore, Huffman coding is particularly useful when data has a **non-uniform frequency distribution**.

---

12. Conclusion

For the given frequencies, fixed-length coding requires **300 bits**, whereas Huffman coding requires only **224 bits**. Thus, Huffman coding saves **76 bits**, giving approximately **25.33% compression** compared with fixed-length coding.

Hence, **Huffman coding is more efficient for this dataset** because the symbol frequencies are highly unequal, especially the very high frequency of F. It is an effective lossless compression technique for real-world applications where data contains frequently occurring patterns.